

1. Plan signal π — JSON shape

1.1 High-level structure

json

```
{
  "plan_id": "uuid",
  "lane_id": "nova-lane-01",
  "timestamp": "2026-06-15T21:55:00Z",
  "source": {
    "type": "user_request | system_event | scheduled_job",
    "id": "req-1234"
  },
  "context": {
    "substrate_state_hash": "sha256:...",
    "ul_version": "ul-2026-06-15.1",
    "rsl_profile": "default-safety-v3"
  },
  "operations": [
    {
      "op_id": "op-1",
      "op_type": "read | write | call | spawn | test | deploy",
      "target": {
        "kind": "file | api | process | repo | env",
        "ref": "path-or-uri-or-id"
      },
      "params": {
        "args": {},
        "metadata": {}
      },
      "preconditions": [
        {
          "kind": "exists | clean_git | tests_pass | capability_present",
          "expr": "domain-specific expression",
          "confidence": 0.95
        }
      ],
      "postconditions": [
        {
          "kind": "file_written | tests_pass | status_code",
          "expr": "domain-specific expression"
        }
      ],
      "dependencies": ["op-0"],
```

```

    "annotations": {
      "from_cortex_trace": "trace-id-or-span",
      "explanation": "why this op exists"
    }
  }
]
}

```

Key properties:

- **plan_id**: stable handle for cross-referencing in ledger.
- **operations[]**: flat list with **dependencies** forming a DAG.

2. Plan signal π — Protobuf sketch

proto

```

message Plan {
  string plan_id = 1;
  string lane_id = 2;
  string timestamp = 3;

  message Source {
    enum Type {
      USER_REQUEST = 0;
      SYSTEM_EVENT = 1;
      SCHEDULED_JOB = 2;
    }
    Type type = 1;
    string id = 2;
  }
  Source source = 4;

  message Context {
    string substrate_state_hash = 1;
    string ul_version = 2;
    string rsl_profile = 3;
  }
  Context context = 5;

  message Condition {
    string kind = 1;
    string expr = 2;
    double confidence = 3; // optional, used mainly for preconditions
  }
}

```

```

message Target {
  string kind = 1; // file, api, process, repo, env
  string ref = 2;
}

message Operation {
  string op_id = 1;
  string op_type = 2; // read, write, call, spawn, test, deploy
  Target target = 3;
  map<string, string> params = 4;
  repeated Condition preconditions = 5;
  repeated Condition postconditions = 6;
  repeated string dependencies = 7;
  map<string, string> annotations = 8;
}

repeated Operation operations = 6;
}

```

3. Ledger event *lt* — JSON shape

This is the **replay atom**: one control-loop step.

```

json
{
  "event_id": "uuid",
  "lane_id": "nova-lane-01",
  "timestamp": "2026-06-15T21:55:01Z",

  "context": {
    "substrate_state_hash_before": "sha256:...",
    "substrate_state_hash_after": "sha256:...",
    "ul_version": "ul-2026-06-15.1",
    "rsl_profile": "default-safety-v3",
    "voss_version": "voss-1.2.0"
  },

  "input": {
    "external": {
      "type": "user_request",
      "id": "req-1234"
    },
  },
}

```

```
"plan_ref": "plan-uuid",
"plan_snapshot": { /* embedded Plan (optional if ref is resolvable) */ }
},
```

```
"constraint_projection": {
  "geometry_id": "ul-compiled-abc123",
  "operations_before": ["op-1", "op-2", "op-3"],
  "operations_after": ["op-1", "op-3"],
  "denied_operations": [
    {
      "op_id": "op-2",
      "reason": "forbidden_region",
      "ul_rule_id": "rule-45"
    }
  ],
  "transforms": [
    {
      "op_id": "op-3",
      "transform_kind": "path_rewrite",
      "details": {
        "from": "/etc/shadow",
        "to": "/sandbox/etc/shadow"
      }
    }
  ]
},
```

```
"safety_decision": {
  "decision": "admit | deny | escalate | wait",
  "rsl_rule_ids": ["rsl-12", "rsl-99"],
  "explanation": "short machine-readable reason",
  "escalation_target": "human | supervisor_lane | quarantine",
  "severity": "info | warn | critical"
},
```

```
"execution": {
  "actions": [
    {
      "op_id": "op-1",
      "status": "success | failure | skipped",
      "side_effects": {
        "files_written": ["path1", "path2"],
        "tests_run": ["test_a", "test_b"],
        "api_calls": ["svc://foo/bar"]
      }
    }
  ]
}
```

```

    },
    "result_hash": "sha256:...",
    "error": {
      "code": "E_IO",
      "message": "Permission denied"
    }
  }
]
},

"meta": {
  "cortex_trace_id": "trace-xyz",
  "voss_node_id": "voss-node-01",
  "signatures": {
    "substrate_sig": "sig-...",
    "voss_sig": "sig-..."
  }
}
}
}

```

Replay guarantee:

Given:

- `substrate_state_hash_before`
- `plan_snapshot` (or resolvable `plan_ref`)
- `constraint_projection`
- `safety_decision`
- `execution.actions`

...you can deterministically reconstruct the state transition and verify it matches `substrate_state_hash_after`.

4. Ledger event ℓt — Protobuf sketch

proto

```

message LedgerEvent {
  string event_id = 1;
  string lane_id = 2;
  string timestamp = 3;

  message Context {
    string substrate_state_hash_before = 1;
    string substrate_state_hash_after = 2;
  }
}

```

```

    string ul_version = 3;
    string rsl_profile = 4;
    string voss_version = 5;
}
Context context = 4;

message ExternalInput {
    string type = 1; // user_request, system_event, scheduled_job
    string id = 2;
}

message Input {
    ExternalInput external = 1;
    string plan_ref = 2;
    Plan plan_snapshot = 3; // optional
}
Input input = 5;

message DeniedOp {
    string op_id = 1;
    string reason = 2;
    string ul_rule_id = 3;
}

message Transform {
    string op_id = 1;
    string transform_kind = 2;
    map<string, string> details = 3;
}

message ConstraintProjection {
    string geometry_id = 1;
    repeated string operations_before = 2;
    repeated string operations_after = 3;
    repeated DeniedOp denied_operations = 4;
    repeated Transform transforms = 5;
}
ConstraintProjection constraint_projection = 6;

message SafetyDecision {
    string decision = 1; // admit, deny, escalate, wait
    repeated string rsl_rule_ids = 2;
    string explanation = 3;
    string escalation_target = 4; // human, supervisor_lane, quarantine

```

```

    string severity = 5; // info, warn, critical
}
SafetyDecision safety_decision = 7;

message Action {
    string op_id = 1;
    string status = 2; // success, failure, skipped
    map<string, string> side_effects = 3;
    string result_hash = 4;

    message Error {
        string code = 1;
        string message = 2;
    }
    Error error = 5;
}

message Execution {
    repeated Action actions = 1;
}
Execution execution = 8;

message Meta {
    string cortex_trace_id = 1;
    string voss_node_id = 2;
    map<string, string> signatures = 3;
}
Meta meta = 9;
}

```

1. Minimal required fields for v1

1.1 Plan π_t (Plan)

Required (v1):

- plan_id : stable ID
- lane_id : which governed lane
- timestamp : ISO8601
- context :
 - $\text{substrate_state_hash}$
 - ul_version
 - rsl_profile

- `operations[]`:
 - `op_id`
 - `op_type` (`read|write|call|spawn|test|deploy`)
 - `target.kind` (`file|api|process|repo|env`)
 - `target.ref` (`path/URI/ID`)
 - `dependencies[]` (may be empty)

Everything else (pre/postconditions, annotations, params) is optional in v1.

1.2 Ledger event `ℓ_t` (LedgerEvent)

Required (v1):

- `event_id`
- `lane_id`
- `timestamp`
- `context`:
 - `substrate_state_hash_before`
 - `substrate_state_hash_after`
 - `ul_version`
 - `rsl_profile`
 - `voss_version`
- `input`:
 - `external.type`
 - `external.id`
 - `plan_ref` or `plan_snapshot` (one must be present)
- `constraint_projection`:
 - `geometry_id`
 - `operations_before[]` (list of `op_id`)
 - `operations_after[]` (list of `op_id`)
- `safety_decision`:
 - `decision` (`admit|deny|escalate|wait`)
- `execution`:
 - `actions[]`:
 - `op_id`
 - `status` (`success|failure|skipped`)

That's the minimal replay core: before/after hash, UL version, plan, projection, decision, actions.

2. RSL decision function `D`

We define:

$D: (\text{Plan}, \text{ConstraintProjection}, \text{Context}) \rightarrow \{\text{admit}, \text{deny}, \text{escalate}, \text{wait}\}$

Where:

- **Plan**: minimal `Plan` (above)
- **ConstraintProjection**: minimal `constraint_projection` (above)
- **Context**: `context` from `LedgerEvent` (UL version, RSL profile, hashes)

3. Hard invariants for D

Let:

- $B = \text{operations_before}$
- $A = \text{operations_after}$

Invariant 1 — Non-empty projected work

If there is nothing left to do, don't admit:

- If $|A| = 0 \rightarrow \text{decision} \neq \text{admit}$
 - Prefer: `deny` (if any ops were denied) or `wait` (if plan is structurally empty).

Invariant 2 — No unknown operations

If `Plan.operations` contains any `op_id` not in B , projection is inconsistent:

- If $\exists \text{ op_id} \in \text{Plan.operations} : \text{op_id} \notin B \rightarrow \text{decision} = \text{escalate}$

Invariant 3 — Hash continuity

If `substrate_state_hash_before` does not match the actual substrate hash at evaluation time:

- $\rightarrow \text{decision} = \text{wait}$ (or `escalate` under stricter profile)

Invariant 4 — UL version consistency

If `Plan.context.ul_version` $\neq$ `Context.ul_version`:

- Under **strict** RSL profile $\rightarrow \text{decision} = \text{escalate}$
- Under **lenient** profile $\rightarrow \text{decision} = \text{deny}$ and require re-plan under current UL.

Invariant 5 — Safety profile presence

If `rsl_profile` is missing or unknown:

- → `decision = escalate` (system misconfiguration)

4. Decision rules (priority-ordered)

Given (`Plan`, `ConstraintProjection`, `Context`):

1. Structural / integrity checks

- If any of:
 - Missing required fields (per v1 minimal set)
 - Hash mismatch
 - UL version mismatch (strict profile)
 - Unknown `rsl_profile`
- → `decision = escalate`

2. Empty or fully-denied plan

- If $|B| > 0 \wedge |A| = 0$
 - Some work was proposed, all removed by constraints
 - → `decision = deny`
- If $|B| = 0$
 - No work proposed
 - → `decision = wait` (quiescent)

3. Safety envelope checks (RSL rules)

Here RSL evaluates domain-specific invariants over `Plan.operations` restricted to `A`:

- Examples:
 - Writes to protected paths (`/etc`, prod secrets)
 - Network calls to forbidden domains
 - Deploys to production without tests in same plan

4. If any **hard safety rule** is violated:

- If rule is **non-escalatory** → `decision = deny`
- If rule is **escalatory** (e.g., prod infra, identity, finance) → `decision = escalate`

5. Normal case

- If all above pass → `decision = admit`

5. Compact pseudo-code for D

pseudo

function D(plan, projection, context):

1. Integrity

if missing_required_fields(plan, projection, context):
 return ESCALATE

```
if !hash_matches_runtime(context.substrate_state_hash_before):  
    return WAIT # or ESCALATE under strict profile
```

```
if plan.context.ul_version != context.ul_version:  
    if is_strict_profile(context.rsl_profile):  
        return ESCALATE  
    else:  
        return DENY
```

```
# 2. Empty / fully denied  
B = projection.operations_before  
A = projection.operations_after
```

```
if len(B) == 0:  
    return WAIT
```

```
if len(A) == 0:  
    return DENY
```

```
# 3. Safety rules  
ops = filter_ops(plan.operations, A)
```

```
safety_result = evaluate_rsl_rules(ops, context)
```

```
if safety_result.hard_violation and safety_result.escalatory:  
    return ESCALATE
```

```
if safety_result.hard_violation:  
    return DENY
```

```
# 4. Normal  
return ADMIT
```

Minimal replay algorithm (v1) and hash continuity

We'll define the **smallest algorithm** that, given `LedgerEvent` v1 + an initial substrate state, can:

- Reconstruct the state transition, and
- Verify that the resulting hash equals `substrate_state_hash_after`.

1. Inputs and assumptions

Inputs:

- Initial substrate state S_0
- Ordered sequence of ledger events $\ell_0, \ell_1, \dots, \ell_n$
 - Each `LedgerEvent` satisfies the v1 minimal field set.

Assumptions:

- There exists a deterministic function:

$H: S \rightarrow \text{hash}$

such that $H(S) = \text{substrate_state_hash_}^*$.

- There exists a deterministic executor:

$E: (S, \text{Action}[]) \rightarrow S'$

that, given a state and a list of actions (`execution.actions`), produces a new state.

- `LedgerEvent.context.substrate_state_hash_before` was computed as $H(S_t)$ at runtime.

2. Minimal replay algorithm

We replay **one event** first, then generalize.

2.1 Single-event replay

Given:

- Substrate state S_t
- Ledger event ℓ_t

Algorithm:

Verify pre-hash

pseudo

```
if  $H(S_t) \neq \text{context.substrate\_state\_hash\_before}$ :  
    fail("Pre-state hash mismatch")
```

- 1.
2. **Resolve plan (optional for v1)**
 - If `input.plan_snapshot` present $\rightarrow$ use it.
 - Else $\rightarrow$ fetch `Plan` by `input.plan_ref`.

- For minimal replay, we don't need full **Plan**—only to ensure that every `execution.actions[ ].op_id` is consistent with `constraint_projection.operations_after[ ]`.

pseudo

`A = projection.operations_after # list of op_id`

for action in `execution.actions`:

 if action.op_id not in A:

 fail("Action executed for non-admitted op_id")

3.

Apply actions

Build the action list:

pseudo

`actions = execution.actions`

Then:

pseudo

`S_t_plus_1 = E(S_t, actions)`

4.

Verify post-hash

pseudo

if `H(S_t_plus_1) != context.substrate_state_hash_after`:

 fail("Post-state hash mismatch")

5.

Return new state

pseudo

return `S_t_plus_1`

6.

If all checks pass, `ℓ_t` is **replay-consistent**.

2.2 Multi-event replay

Given:

- Initial state `S0`
- Sequence `ℓ0...ℓn`

Algorithm:

```

pseudo
S = S_0

for t in 0..n:
    # 1. Pre-hash check
    if H(S) !=  $\ell_t$ .context.substrate_state_hash_before:
        fail("Pre-hash mismatch at event t")

    # 2. Admitted ops consistency
    A =  $\ell_t$ .constraint_projection.operations_after
    for action in  $\ell_t$ .execution.actions:
        if action.op_id not in A:
            fail("Executed non-admitted op at event t")

    # 3. Apply actions
    S = E(S,  $\ell_t$ .execution.actions)

    # 4. Post-hash check
    if H(S) !=  $\ell_t$ .context.substrate_state_hash_after:
        fail("Post-hash mismatch at event t")

# If loop completes:
return S

```

3. Hash continuity invariant (formal statement)

For each $t \in [0, n]$:

1. Pre-state consistency

$H(S_t) = \ell_t$.context.substrate_state_hash_before

2. Post-state consistency

Let:

$S_{t+1} = E(S_t, \ell_t$.execution.actions)

Then:

$H(S_{t+1}) = \ell_t$.context.substrate_state_hash_after

3. Sequential continuity

ℓ_t .context.substrate_state_hash_after = ℓ_{t+1} .context.substrate_state_hash_before

If all three hold for every t , the ledger sequence is **deterministically replayable** and the substrate evolution is **fully verified**.

4. What this gives you (in one line)

Given an initial state and a v1 ledger, you can:

- Recompute every state transition,
- Prove that each transition matches the recorded hashes, and
- Detect any tampering, omission, or non-deterministic behavior as a hash continuity violation.

v1 RSL rule set — safety as first-class, replayable events

Here's a **tight v1 rule pack** that maps cleanly onto your existing signals and ledger.

1. Rule catalog (names + intent)

NO_PROD_WRITE:

- **Intent:** Prevent any write to production-critical surfaces without explicit, governed override.

NO_SECRET_READ:

- **Intent:** Block direct access to secrets (keys, tokens, credentials) from governed lanes.

NO_UNTESTED_DEPLOY:

- **Intent:** Forbid deploy operations unless tests have passed in the same or prior verified plan.

NO_DIRECT_IDENTITY_MUTATION:

- **Intent:** Prevent direct changes to identity records (users, roles, permissions) without escalation.

NO_UNSCOPED_NETWORK_CALL:

- **Intent:** Block network calls to endpoints not explicitly whitelisted in UL.

NO_SELF_MODIFYING_RUNTIME:

- **Intent:** Forbid writes to the governed runtime itself (CoG-OS, Voss, RSL, UL artifacts).

These six give you a strong v1 safety lattice without overcomplicating.

2. Rule wiring — how each sees Plan + Context

Each rule evaluates over:

- `Plan.operations[ ]` (restricted to `operations_after` from `ConstraintProjection`)
- `Context.ul_version / rsl_profile`
- Optional: domain config (e.g., prod paths, secret stores, identity DBs, allowed endpoints)

I'll sketch each as a predicate:

2.1 NO_PROD_WRITE

Scope: `op_type = "write"` and `target.kind = "file|repo|env"`

pseudo

```
if op_type == "write" and is_prod_target(target.ref):  
    return HARD_VIOLATION, ESCALATORY
```

- `is_prod_target`: matches paths like `/etc`, `/var/lib/prod`, prod repos, prod envs.

2.2 NO_SECRET_READ

Scope: `op_type = "read"` and `target.kind = "file|api|env"`

pseudo

```
if op_type == "read" and is_secret_surface(target.ref):  
    return HARD_VIOLATION, ESCALATORY
```

- `is_secret_surface`: secret vaults, `.env` with secrets, KMS APIs, credential stores.

2.3 NO_UNTESTED_DEPLOY

Scope: `op_type = "deploy"`

pseudo

```
if op_type == "deploy" and !tests_passed_in_plan_or_recent(plan_id, lane_id):  
    return HARD_VIOLATION, NON_ESCALATORY
```

- `tests_passed_in_plan_or_recent` checks:
 - Same `plan_id` has `op_type = "test"` with success, or
 - Recent ledger events for this `lane_id` show passing tests on same artifact.

2.4 NO_DIRECT_IDENTITY_MUTATION

Scope: `op_type = "write|call"` and `target.kind = "api|repo"`

pseudo

```
if touches_identity_surface(target.ref):  
    return HARD_VIOLATION, ESCALATORY
```

- `touches_identity_surface` matches IAM APIs, user DBs, role config repos.

2.5 NO_UNSCOPED_NETWORK_CALL

Scope: `op_type = "call"` and `target.kind = "api"`

pseudo

```
if is_network_endpoint(target.ref) and !is_whitelisted_endpoint(target.ref, ul_version):  
    return HARD_VIOLATION, NON_ESCALATORY
```

- UL carries the whitelist; rule enforces it.

2.6 NO_SELF_MODIFYING_RUNTIME

Scope: `op_type = "write|deploy"` and `target.kind = "file|repo|env"`

pseudo

```
if is_runtime_surface(target.ref): # CoG-OS, Voss, RSL, UL, core binaries  
    return HARD_VIOLATION, ESCALATORY
```

3. Aggregation into RSL decision D

We define:

pseudo

```
function evaluate_rsl_rules(ops, context):
```

```
    violations = []
```

```
    for op in ops:
```

```
        for rule in [NO_PROD_WRITE,  
                     NO_SECRET_READ,  
                     NO_UNTESTED_DEPLOY,  
                     NO_DIRECT_IDENTITY_MUTATION,  
                     NO_UNSCOPED_NETWORK_CALL,
```

NO_SELF_MODIFYING_RUNTIME]:

```
v = rule(op, context)
if v.is_hard_violation:
    violations.append(v)

if violations.empty:
    return { hard_violation: false }

if any(v.escalatory for v in violations):
    return { hard_violation: true, escalatory: true, violations }

return { hard_violation: true, escalatory: false, violations }
```

Then your existing D logic applies:

- `hard_violation && escalatory → decision = escalate`
- `hard_violation && !escalatory → decision = deny`

Each violation is logged into `LedgerEvent.safety_decision.rsl_rule_ids` and, if you want, a dedicated `violations[ ]` block for forensic clarity.

4. Why this v1 rule set is enough

With just these:

- You can't silently write to prod.
- You can't silently read secrets.
- You can't deploy without tests.
- You can't mutate identity directly.
- You can't call random network endpoints.
- You can't rewrite the runtime that governs you.

That's a **real safety lattice**, not a vibes checklist.

NO_PROD_WRITE — UL encoding as constraint geometry

Let's turn **NO_PROD_WRITE** from "a rule" into **UL law** that both Voss and RSL consume as geometry.

1. Law intent (semantic form)

Human-level statement:

Writes to production surfaces are forbidden for this lane unless explicitly authorized by a higher law and routed through an override path.

We'll encode:

- What counts as “production surface”
- What “write” means in UL terms
- How exceptions are expressed (if any)

2. UL law packet — symbolic sketch

Assume UL has a structured form like:

```
ul
law NO_PROD_WRITE {
  scope: lane("nova-lane-01")

  surfaces.prod {
    file: [
      "/etc/**",
      "/var/lib/prod/**",
      "/opt/prod/**"
    ],
    repo: [
      "git://prod/**",
      "repo://infra/prod/**"
    ],
    env: [
      "ENV:PROD_*"
    ]
  }

  forbid op.write on surfaces.prod
}
```

Core pieces:

- **scope**: which lane(s) this law applies to.
- **surfaces.prod**: named geometry—paths, repos, env keys.
- **forbid op.write on surfaces.prod**: the actual constraint.

3. Compilation to constraint geometry

UL compiler turns this into a **geometry object** Cprod:

Cprod={file_patterns,repo_patterns,env_patterns}

Example:

```
json
{
  "id": "ul-geom-prod-write-001",
  "surfaces": {
    "file": ["/etc/**", "/var/lib/prod/**", "/opt/prod/**"],
    "repo": ["git://prod/**", "repo://infra/prod/**"],
    "env": ["ENV:PROD_*"]
  },
  "constraints": [
    {
      "op_type": "write",
      "target_kinds": ["file", "repo", "env"],
      "effect": "deny",
      "law_id": "NO_PROD_WRITE"
    }
  ]
}
```

This is what Voss and RSL actually see: **pattern geometry + constraint effect**.

4. Voss consumption (constraint projection)

In the Voss loop, for each operation ok:

pseudo

function G_C(o_k):

if o_k.op_type != "write":
 return ALLOWED

if o_k.target.kind == "file" and matches_any(o_k.target.ref, surfaces.file):
 return DENIED

if o_k.target.kind == "repo" and matches_any(o_k.target.ref, surfaces.repo):
 return DENIED

if o_k.target.kind == "env" and matches_any(o_k.target.ref, surfaces.env):
 return DENIED

return ALLOWED

Result:

- Any **write** to a prod surface is **removed at projection time** (π_t), never reaches execution.

5. RSL consumption (safety lattice)

RSL sees the **same geometry**, but as a **safety rule**:

pseudo

```
rule NO_PROD_WRITE_RSL(op, context):
```

```
    if op.op_type != "write":
        return NO_VIOLATION
```

```
    if is_prod_surface(op.target):
        return HARD_VIOLATION, ESCALATORY, "NO_PROD_WRITE"
```

So:

- Voss **clips** the plan (no such writes admitted).
- RSL **flags** any attempt as a safety violation, even if somehow a write slipped through a misconfig.

Both are driven by the **same UL geometry**.

6. Why this matters structurally

You've now:

- Defined **prod surfaces once** (in UL).
- Compiled them into **constraint geometry**.
- Fed that geometry into:
 - **Voss** (projection: "this will never execute"), and
 - **RSL** (safety: "this attempt is a violation and must be logged/escalated").

Law isn't prose anymore—it's **shape** that the control system runs on.

NO_SECRET_READ — UL as "no-cut" zone around secrets

Let's make **NO_SECRET_READ** just as physical as "don't crash the spindle into the vise."

1. Law intent

Human statement:

Reads from secret surfaces (keys, tokens, credentials, vaults) are forbidden for this lane, unless explicitly routed through a governed secret-access mechanism.

We'll encode:

- What counts as a **secret surface**
- What “read” means
- How exceptions could be carved out later (but v1: none)

2. UL law packet — symbolic sketch

ul

```
law NO_SECRET_READ {  
  scope: lane("nova-lane-01")
```

```
  surfaces.secrets {  
    file: [  
      "/etc/ssl/private/**",  
      "/var/lib/secrets/**",  
      "**/.env.secrets",  
      "**/secrets.yml"  
    ],  
    api: [  
      "svc://vault/*",  
      "https://secrets.internal/*"  
    ],  
    env: [  
      "ENV:AWS_*",  
      "ENV:SECRET_*",  
      "ENV:TOKEN_*"  
    ]  
  }  
}
```

```
  forbid op.read on surfaces.secrets  
}
```

- `surfaces.secrets`: the “no-cut” volume around secrets.
- `forbid op.read on surfaces.secrets`: the constraint.

3. Compiled constraint geometry

UL compiler emits a geometry object Csecrets:

```
json
{
  "id": "ul-geom-secret-read-001",
  "surfaces": {
    "file": [
      "/etc/ssl/private/**",
      "/var/lib/secrets/**",
      "**/.env.secrets",
      "**/secrets.yml"
    ],
    "api": [
      "svc://vault/*",
      "https://secrets.internal/*"
    ],
    "env": [
      "ENV:AWS_*",
      "ENV:SECRET_*",
      "ENV:TOKEN_*"
    ]
  },
  "constraints": [
    {
      "op_type": "read",
      "target_kinds": ["file", "api", "env"],
      "effect": "deny",
      "law_id": "NO_SECRET_READ"
    }
  ]
}
```

Same pattern as NO_PROD_WRITE: **patterns + effect**.

4. Voss consumption — projection

In Voss:

```
pseudo
function G_C_secret(o_k):

  if o_k.op_type != "read":
    return ALLOWED
```

```

if o_k.target.kind == "file" and matches_any(o_k.target.ref, surfaces.file):
    return DENIED

if o_k.target.kind == "api" and matches_any(o_k.target.ref, surfaces.api):
    return DENIED

if o_k.target.kind == "env" and matches_any(o_k.target.ref, surfaces.env):
    return DENIED

return ALLOWED

```

Effect:

- Any **read** aimed at a secret surface is **clipped out of the plan**.
- It never becomes an executable action.

This is the CNC equivalent of: *the toolpath generator simply never emits moves that intersect the machine bed.*

5. RSL consumption — safety lattice

RSL uses the same geometry as a **hard safety rule**:

pseudo

```
rule NO_SECRET_READ_RSL(op, context):
```

```

    if op.op_type != "read":
        return NO_VIOLATION

```

```

    if is_secret_surface(op.target): # same pattern matcher as Voss
        return HARD_VIOLATION, ESCALATORY, "NO_SECRET_READ"

```

So:

- If Cortex even *tries* to plan a secret read,
- Voss drops it,
- RSL logs it as a **safety violation** and can escalate (e.g., to a human or supervisor lane).

6. How this becomes “physically real”

With NO_SECRET_READ encoded this way:

- **Cortex cannot “accidentally” read secrets**—the geometry forbids it.

- **Voss will never emit a secret read as motion**—the toolpath is clipped.
- **RSL will always record and escalate attempts**—violations are first-class in the ledger.

Secrets become a **no-cut volume** in the cognitive workspace—just like the machine bed or fixture in a CNC: always present, always respected, enforced by geometry, not vibes.

1. Concept: Secret Proxy as the only legal tool

Idea:

No lane ever reads secrets directly.

All secret access is:

- **Isolated to a dedicated lane** (Secret Proxy lane)
- **Mediated by a narrow tool API**
- **Fully logged and policy-checked**

So “read secret” becomes:

Request to Secret Proxy → Proxy decides → Proxy returns redacted/derived value

2. Architecture: governed secret-access lane

2.1 Lanes

- **Primary lane (Law-Bound Coder):**
 - **Forbidden:** direct **read** on secret surfaces (NO_SECRET_READ).
 - **Allowed:** **call** to **tool://secret-proxy/get**.
- **Secret Proxy lane:**
 - Has **capability** to read from secret surfaces.
 - Runs under **stricter UL + RSL profile**.
 - Only exposes **high-level operations**, not raw dumps.

2.2 UL wiring

1. Primary lane UL:

```
ul
law NO_SECRET_READ {
  scope: lane("coder-lane")

  surfaces.secrets { ... }

  forbid op.read on surfaces.secrets
}
```

```

law SECRET_PROXY_ONLY {
  scope: lane("coder-lane")

  allow op.call on api("tool://secret-proxy/*")
}

```

2. Secret Proxy lane UL:

```

ul
law SECRET_PROXY_CAPS {
  scope: lane("secret-proxy-lane")

  allow op.read on surfaces.secrets
  forbid op.write on surfaces.secrets
}

```

3. Secret Proxy tool interface

Define a **narrow, typed API**:

```

json
// Request from coder lane to secret proxy
{
  "tool": "tool://secret-proxy/get",
  "request_id": "uuid",
  "caller_lane": "coder-lane",
  "purpose": "deploy_to_env",
  "secret_ref": "env:PROD_DB_PASSWORD",
  "scope": {
    "service": "billing-api",
    "env": "staging"
  }
}

```

Proxy response:

```

json
{
  "request_id": "uuid",
  "status": "ok | denied",
  "reason": "policy_violation | ok",
  "value": "*****", // maybe redacted
}

```

```
"token_ref": "secret-token-123" // or short-lived handle
}
```

Key: the coder lane never sees the raw vault path or full secret set—only what the proxy chooses to reveal.

4. RSL rules for the Secret Proxy lane

Examples:

- **NO_BULK_SECRET_DUMP**
 - Deny requests without a specific `secret_ref`.
- **NO_PROD_SECRET_FOR_NON_PROD_SCOPE**
 - Deny if `secret_ref` is prod but `scope.env != prod`.
- **NO_SECRET_RETURN_WITHOUT_PURPOSE**
 - Deny if `purpose` is missing or not in an allowlist.

These run **inside the Secret Proxy lane**, on its own RSL profile.

5. Ledger semantics

Every secret access becomes a **first-class ledger event** in the Secret Proxy lane:

- `external.type = "tool_call"`
- `input.external.id = "coder-lane:request-id"`
- `execution.actions[].op_type = "read"`
- `target.ref = secret surface`
- `safety_decision` shows:
 - Which RSL rules evaluated
 - Whether access was admitted/denied
- Optional: `meta.redaction_policy` to show how the value was shaped.

You now have:

- **Coder lane ledger:** “I asked Secret Proxy for X.”
- **Secret Proxy ledger:** “I checked law, read secret Y, and returned Z under policy P.”

6. What this buys you

- Secrets are **never touched directly** by general cognition.
- All secret access is:
 - **Isolated** (separate lane)
 - **Constrained** (narrow tool API)

- **Audited** (own ledger stream)
- **Governed by UL + RSL** specific to secrets.

That's exactly like a CNC where:

- The main program can't move the spindle into the fixture.
- Only a guarded, interlocked macro can change the fixture—and every use is logged.

1. Global default: **NO_SECRET_READ** everywhere

First, you set the **baseline**: no lane can read secrets.

```
ul
law NO_SECRET_READ_GLOBAL {
  scope: lanes("") // all lanes by default

  surfaces.secrets {
    file: [
      "/etc/ssl/private/**",
      "/var/lib/secrets/**",
      "**/.env.secrets",
      "**/secrets.yml"
    ],
    api: [
      "svc://vault/**",
      "https://secrets.internal/**"
    ],
    env: [
      "ENV:AWS_**",
      "ENV:SECRET_**",
      "ENV:TOKEN_"
    ]
  }

  forbid op.read on surfaces.secrets
}
```

This compiles to the **secret geometry** every lane inherits.

2. Designate the only exception lane

Now you carve out **one lane** that's allowed to cross that geometry:

```
ul
```

```

law SECRET_PROXY_EXCEPTION {
  scope: lane("secret-proxy-lane")

  allow op.read on surfaces.secrets
  forbid op.write on surfaces.secrets
}

```

Key points:

- It **reuses the same** `surfaces.secrets` **geometry** from the global law.
- It **overrides only for that lane**, and only for `op.read`.
- It still forbids writes, so the proxy can't mutate the secret store.

Precedence rule (implicit in UL model):

- **Specific lane law > global law**, when both talk about the same surface and op.

3. Enforce “everyone else must use the proxy”

For the **coder lane** (and others), you make the path explicit:

```

ul
law SECRET_ACCESS_VIA_PROXY_ONLY {
  scope: lane("coder-lane")

  // Already inherited:
  // forbid op.read on surfaces.secrets (from NO_SECRET_READ_GLOBAL)

  allow op.call on api("tool://secret-proxy/*")
}

```

So for `coder-lane`:

- Direct `read` on secrets → **forbidden by global law**.
- `call` to `tool://secret-proxy/get` → **explicitly allowed**.

4. How this looks in constraint geometry

After compilation you effectively have:

- **Global geometry** Csecrets:
 - `read` on secret surfaces → `deny` for all lanes.
- **Override geometry** for `secret-proxy-lane`:

- `read` on secret surfaces → `allow` (wins by specificity).
- `write` on secret surfaces → `deny`.
- **Coder lane geometry:**
 - `read` on secret surfaces → `deny`.
 - `call` to `tool://secret-proxy/*` → `allow`.

No other lane gets an override, so **the only legal path across the secret boundary is:**

```
coder-lane → call tool://secret-proxy/* → secret-proxy-lane →
read surfaces.secrets
```

5. One-line UL doctrine

NO_SECRET_READ is universal law; **SECRET_PROXY_EXCEPTION** is the only lawful corridor through it, bound to a single lane and a narrow tool interface.

That's how you get an exception that behaves like a guarded fixture change—not a hole in the fence.